



SJSU CMPE 172

CMPE 172-02 – MILESTONE 1 REPORT

Online Appointment Scheduling System: Interview Scheduler

Name: Charlie Banh

Student ID: 015345473

Date: September 25, 2026

Chosen scenario: Interview Scheduler to help with resume reviews and mock interviews for software engineers

GitHub repository (milestone tag): <https://github.com/c-banh-mi/cmpe172-interview-scheduler/tree/milestone-1>

Code-walkthrough video: <https://drive.google.com/file/d/1bjLsSmwjBthYJkkrbdtsvMnHgxpHTZpi/view?usp=sharing>

1. Overview

Milestone 1 implements the running layered skeleton of this online appointment scheduling system with the initial architecture, database schema, and wireframes. This delivers a system proposal, relational database design with constraints and indexing, UI/UX wireframes, and a functional Spring Boot backend serving core read endpoints through SQL over JDBC.

2. Design and Implementation

2.1 Proposal

Scenario

The system is an interview scheduling platform designed to help connect inexperienced software engineers with experienced mentors and professionals for career development and support. Users can book 1:1 sessions for resume reviews, mock interviews, and interview preparation. An appointment system enforces time-slot availability, prevents double-booking with a partial unique index or database constraint, and maps a many-to-one relationship between bookable windows and mentors' schedules.

User roles

Role	Who they are in this scenario	What they can do
Customer	A software engineer or job seeker looking for interview practice or career guidance	Browse and filter available slots, book appointments, view upcoming or past appointments, and cancel bookings
Provider/Admin	An experienced engineer or other professional acting as a mentor	Log in, create and remove availability slots, and view booked appointments

Core features

#	Feature	Role	Milestone	How it works in my scenario
1	Customer log in	Customer	M2	Authenticates user credentials and issues a session/token
2	Browse, filter, paginate slots (LIMIT/OFFSET)	Customer	M1 read /M2	Queries open availability filtered by service, mentor, or date with pagination
3	Book a slot for a service	Customer	M2	Locks and transitions an open availability slot into a confirmed booking
4	View my appointments (upcoming + history)	Customer	M2	Fetches active and past appointments for the logged-in user
5	Cancel my appointment (owner only)	Customer	M2	Allows a customer to cancel their booking, which frees up the spot
6	Provider log in	Provider	M2	Authenticates mentor credentials
7	Create/remove availability slots	Provider	M2	Mentors publish new time windows or remove unused slots
8	View appointments booked	Provider	M2	Mentors view schedule of confirmed mock interviews and resume reviews
9	Confirmation via mock external service	System	M3	Sends asynchronous notification callbacks via a mock external email or webhook service
10	Logging, health endpoint, metric	System	M3	Exposes actuator metrics, health checks, and structured request logging
11	Conversational + autonomous AI agents	System	M4	An autonomous agent runs a task on its own without a user prompting it, like a scheduled job
12	Grounded (RAG) Q&A over policy/FAQ	System	M4	Retrieval-Augmented Generation answers policy and FAQ questions

Main entities

- Users: store user account details, hashed passwords, emails, and roles (CUSTOMER or PROVIDER)
- Providers: extended mentor profiles linked 1:1 to a user, storing professional headlines, display names, and bio
- Services: define bookable offerings, e.g., mock coding interview, resume review, along with duration and pricing
- availability_slots: time windows published by providers offering specific services, tracking status (OPEN, BOOKED, REMOVED) and optimistic locking versions
- Appointments: record customer bookings linked to specific slots and services, supporting status tracking (BOOKED, CANCELLED, COMPLETED) and user notes

2.2 Architecture

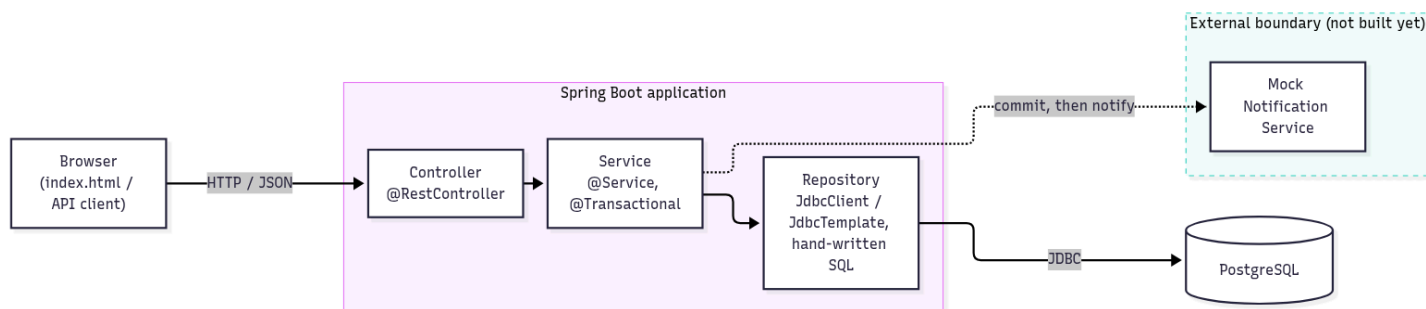


Figure 1. System block diagram

Architectural Breakdown & Data Flow (refer to Figure 1)

- Browser/API Client: sends HTTP requests carrying JSON payloads or query parameters
- Controller (@RestController): handles HTTP routing, parameter extraction, and delegates business logic to the service layer; controllers never contain raw SQL or business rules
- Service (@Service, @Transactional): forms transactional boundaries; by the dotted “commit, then notify” boundary, database transactions commit fully before triggering external notifications to ensure consistency
- Repository (JdbcClient): executes handwritten SQL directly against PostgreSQL without relying on an ORM abstraction layer
- PostgreSQL: stores relational data with ACID

Technology Stack

- Core Framework: Java 21 + Spring Boot 3.5.16
- Data Access: pure SQL via Spring’s JdbcClient/JdbcTemplate (no ORM/Hibernate)
- Enterprise Standards: modernized implementation of classic enterprise architecture patterns, i.e. Spring Boot for J2EE-equivalent enterprise container services, REST for CORBA/distributed objects
- Database: PostgreSQL 16 is chosen over MySQL/SQLite for its capabilities with partial indexes and row-locking mechanisms



2.3 Database design

ER diagram

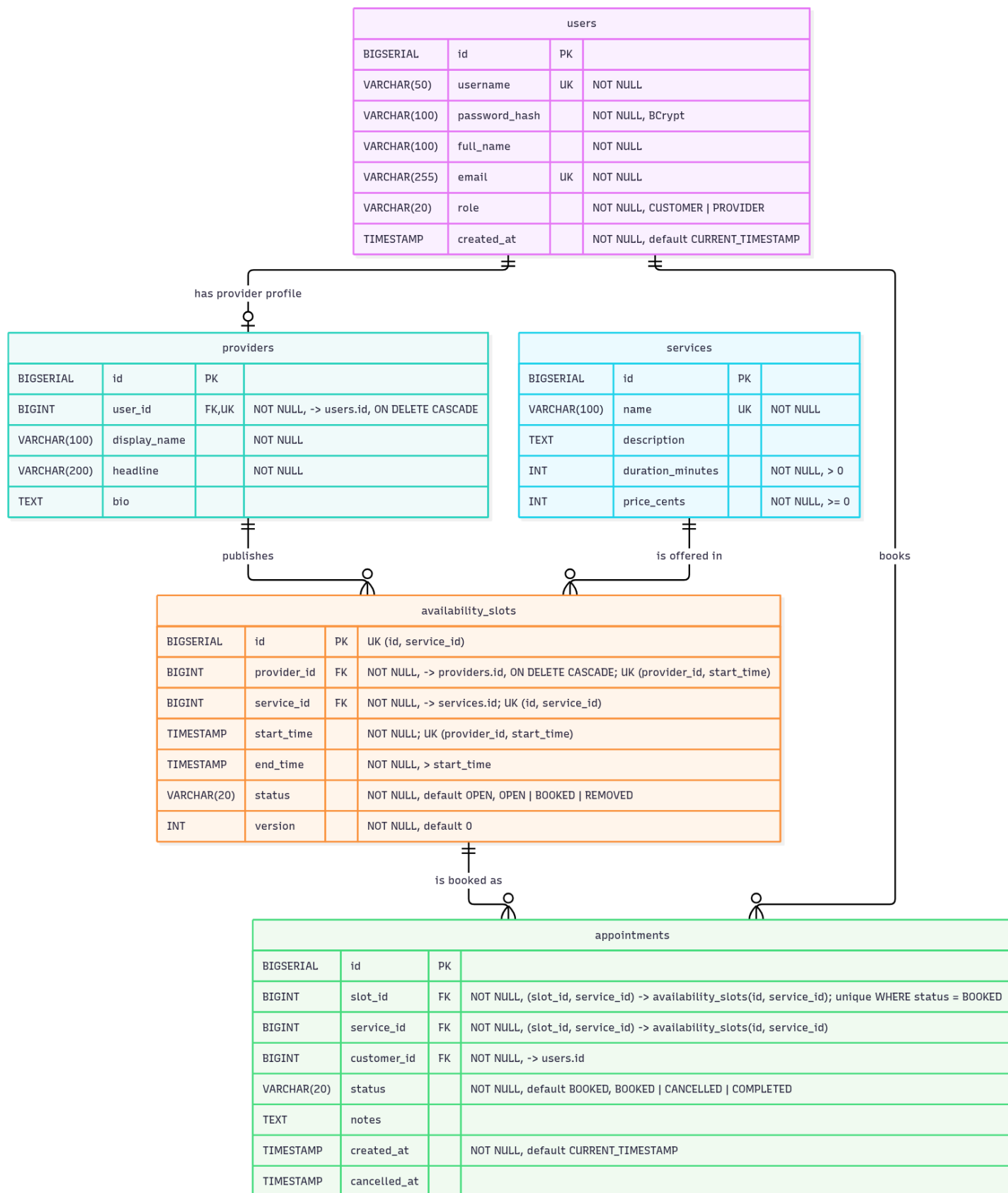


Figure 2. ER diagram

Relational schema

The database uses normalized relational structures with surrogate BIGSERIAL primary keys across all tables. Referential integrity is strictly maintained using foreign keys, composite constraints, and a partial unique index (a database constraint). The Full SQL DDL schema is provided in Appendix A and B.

Keys and relationships

Relationship	Cardinality	Implemented by (FK / constraint)	Why this cardinality
users – providers	1:0..1	providers.user_id FK + UNIQUE	A user can optionally be a mentor/provider; if they are, they have exactly one provider profile
providers – availability_slots	1:N	availability_slots.provider_id FK	A single provider can publish multiple open time slots
services – availability_slots	1:N	availability_slots.service_id FK	A service type can be offered across many different time slots
availability_slots – appointments	1:N	fk_appt_slot_service (slot_id, service_id)	While a slot typically maps to 0..1 active appointments, maintaining a 1:N history allows past cancellations to remain auditable
users – appointments	1:N	appointments.customer_id FK	A customer can book multiple appointments over time

Double-booking guard

- **Mechanism:** CREATE UNIQUE INDEX uq_appt_active_slot ON appointments (slot_id) WHERE status = 'BOOKED';
- **Validation Test:** ApiIntegrationTest.secondActiveBookingForSameSlotIsRejectedByDatabase
- **Explanation:** This partial unique index guarantees that at most one active (BOOKED) appointment can exist for any given time slot. Unlike a standard table-level unique constraint, it permits historical or cancelled appointment rows (status = 'CANCELLED') to remain in the table for audit trails without blocking future re-bookings of the same slot. The composite foreign key (slot_id, service_id) makes the appointment's service match the slot's service. What stops a race condition is the unique index. uq_slot_provider_start ensures that a mentor cannot publish two slots with the same start time

Justification

- **Cardinality:** The users to providers 1:0..1 split cleanly separates base authentication from mentor-specific profile attributes without bloating non-mentor user records. Slots-to-appointments is 1:N to preserve booking history and cancellation records.
- **Weak Entities & Surrogate Keys:** availability_slots and appointments depend on parent entities, and every table uses a surrogate BIGSERIAL primary key (id) to simplify indexing, REST endpoint routing, and foreign key references across application layers.
- **Constraints & Data Integrity:** includes strict CHECK constraints (role IN ('CUSTOMER', 'PROVIDER'), slot statuses, duration_minutes > 0, price_cents >= 0, end_time > start_time), composite foreign keys, cascading deletes (ON DELETE CASCADE on provider relationships), and soft-removal states (REMOVED) for slots

2.4 Wireframes

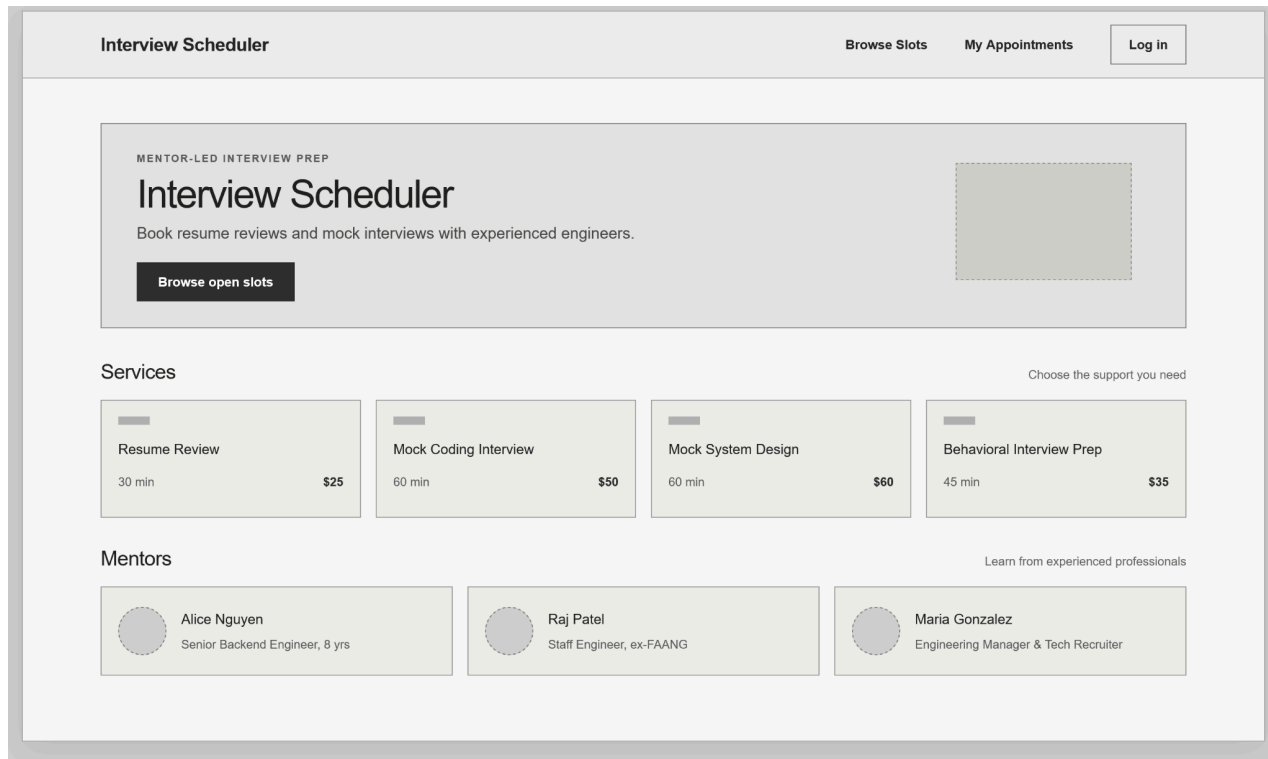


Figure 3. Home (above): Landing page introducing the platform, featured services (Resume Review, Mock Coding, System Design, Behavioral), and top mentors. Maps to GET /api/home.

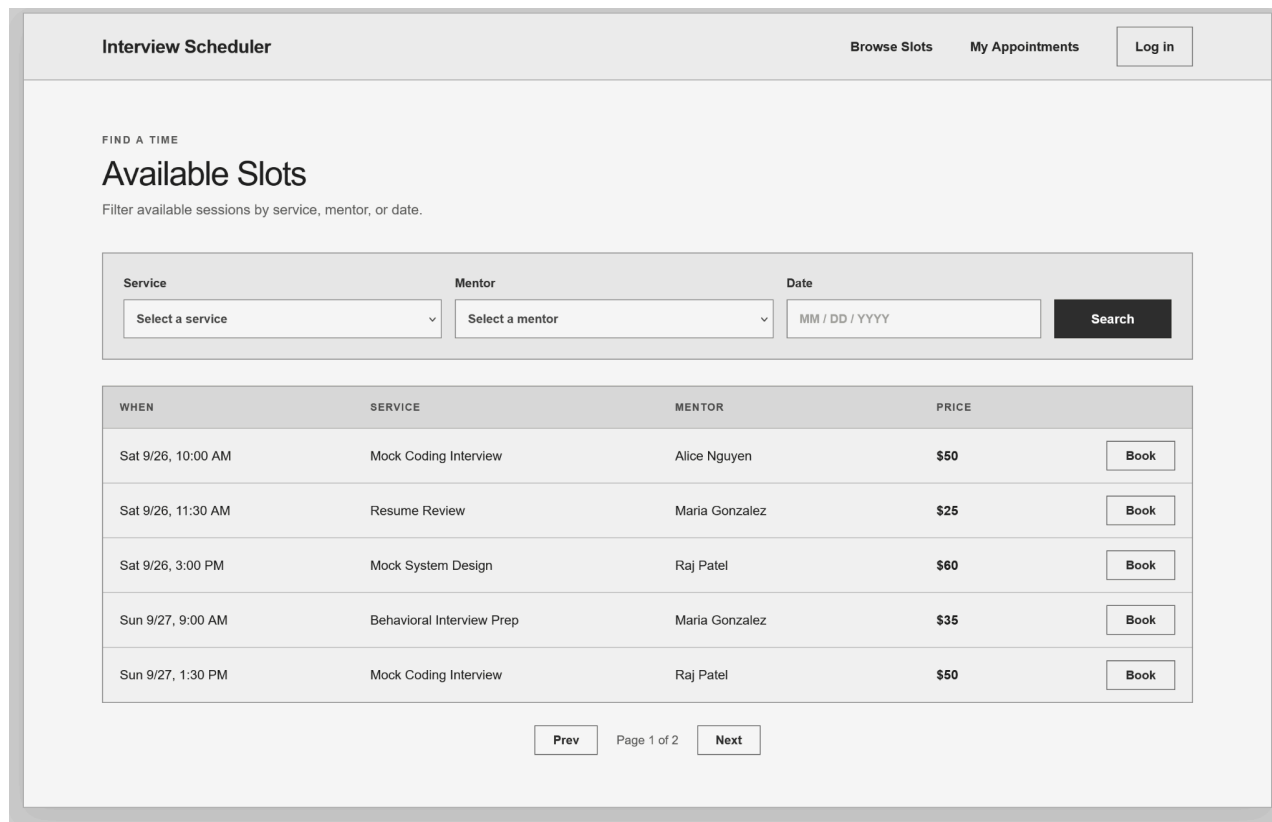


Figure 4. Available slots: Filterable, paginated table of open interview slots by service, mentor, or date with pagination controls (Prev/Next). Maps to GET /api/slots.

Interview Scheduler
Browse Slots
My Appointments
Log in

FINAL STEP

Book Appointment

Review your session details and add a note for your mentor.

APPOINTMENT SUMMARY

SERVICE	Mock System Design
MENTOR	Raj Patel
TIME	Sat 9/26, 3:00–4:00 PM
PRICE	\$60

Notes

Help your mentor prepare for your session.

Target role, resume link, topics to focus on

Confirm Booking
Cancel

Figure 5. Book appointment (above): Final review step where customers verify appointment details and input optional preparation notes before confirming. This is planned for Milestone 2.

Interview Scheduler
Browse Slots
My Appointments
Log in



You're booked!

A confirmation has been sent to your email.

APPOINTMENT DETAILS

APPOINTMENT #1042	SERVICE Mock System Design
MENTOR Raj Patel	TIME Sat 9/26, 3:00–4:00 PM
PRICE \$60	

View My Appointments
Back to Slots

Figure 6. Confirmation: Success screen displaying booked appointment details, mentor, time, and price confirmation.

2.5 Running skeleton

The application boots with pre-loaded seed data (schema.sql and seed.sql configured in application.yml with mode always), serving static web assets alongside REST endpoints.

Method	Path	Reads	Returns (DTO)
GET	/api/home	DB tables: services, providers, COUNT of open slots	HomeResponse
GET	/api/slots	Query tables: availability_slots JOIN providers JOIN services (+ COUNT)	PageResponse<SlotDto>
GET	/api/services	All rows of services	List<ServiceDto>
GET	/api/providers	All registered mentor profiles	List<ProviderDto>

2.6 Request flow through the layers

When a client requests GET /api/slots?serviceId=1&page=0:

1. **DispatcherServlet** intercepts the HTTP request and routes it to the appropriate controller method
2. **SlotController** receives query parameters and invokes the service layer, ensuring controllers never handle raw database entities
3. **SlotService** applies business logic, validation, and converts page parameters into SQL LIMIT and OFFSET values
4. **SlotRepository** uses Spring's JdbcTemplate to execute handwritten SQL parameterized queries against PostgreSQL
5. **PostgreSQL** executes the query and returns raw tabular result rows
6. **Mapping & Response** rows are mapped via RowMapper into domain views (SlotView), transformed into DTOs (SlotDto), wrapped inside a PageResponse, and serialized into JSON for the client

2.7 Page Controller vs Front Controller

A Page Controller handles requests directly within individual page scripts or entry points, and a Front Controller funnels all incoming HTTP requests through a single central controller (DispatcherServlet in Spring Boot) which dispatches requests to appropriate handlers.

This project utilizes the Front Controller pattern via Spring's DispatcherServlet, returning clean JSON REST endpoints consumed by a static client page (index.html). This architecture was chosen over server-rendered Thymeleaf or JSP because it detaches the backend API from frontend rendering, aligning with modern enterprise REST standards and facilitating smoother future expansion into SPAs or mobile clients.

3. Design Decisions and Trade-offs

- **Slot Status Storage:** Slot statuses (OPEN, BOOKED, REMOVED) are stored directly on the availability_slots table alongside appointment statuses.
 - This enables fast filtering during slot browsing (WHERE status = 'OPEN') without requiring expensive outer joins on every slot query. It also requires careful transactional synchronization between slots and appointments to keep both states consistent.
- **Monetary Representation (price_cents INT):** All prices are stored as integer cents rather than floating-point decimals to avoid rounding errors during financial calculations.
- **Spring Boot 3.5.16 over 4.x:** This project opted for a more stable version, avoiding 4.x from breaking anything. The trade-off is that 3.5 is past its open-source end of life.

- **Known Limitations:** redundancy between appointments.service_id and availability_slots.service_id (mitigated by composite foreign keys); slot overlapping by the same mentor is not strictly blocked at the database level (handled via application validation planned for Milestone 2); roles are not enforced through multi-table foreign keys; timestamps are stored without explicit time zones (TIMESTAMP)

4. Challenges and Solutions

- **Timezone Container Discrepancy:**
 - *Challenge:* date filters returned empty results after 5:00 PM Pacific Time due to Docker containers defaulting to UTC time
 - *Solution:* added TZ: America/Los_Angeles to the app services in docker-compose.yml
- **Secret Management:**
 - *Challenge:* preventing database credentials and local configuration secrets from leaking into public git history
 - *Solution:* implemented environment-based configuration loading using .env and .env.example templates

5. Testing and Results

- **Test Execution:** Running mvn -B package successfully compiles the application and executes integration tests, resulting in 8 passing tests.
- **CI:** Clean-clone setup runs successfully following repository instructions, backed by a fully green GitHub Actions CI pipeline.
- **Verification Artifacts (see Appendix C):**
 - Home page UI operational
 - /api/home returning valid JSON payloads
 - /api/slots returning paginated slot listings

6. AI Tools Used

- Claude Code was used to accelerate generation for Spring Boot, JDBC repositories, structure Docker and CI configuration files, assist in debugging timezone discrepancies, and generate Mermaid system diagrams.
- Figma was used to design and generate the four mockup wireframes.

7. Additional

To build and run the project locally, refer to the repository README.md for complete configuration details

Appendix

Appendix A. Relational schema

Source: src/main/resources/schema.sql (PostgreSQL 16). Constraint names without an explicit name in schema.sql are the names PostgreSQL generates.

users

Column	Type	Nullable	Default
id	BIGSERIAL (bigint)	NO	nextval('users_id_seq')
username	VARCHAR(50)	NO	
password_hash	VARCHAR(100)	NO	
full_name	VARCHAR(100)	NO	
email	VARCHAR(255)	NO	
role	VARCHAR(20)	NO	
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP

PK: users_pkey (id)

FKs: none

UNIQUE: users_username_key (username); users_email_key (email)

CHECK: users_role_check: role IN ('CUSTOMER', 'PROVIDER')

Indexes: users_pkey, users_username_key, users_email_key (unique B-tree indexes backing the constraints)

providers

Column	Type	Nullable	Default
id	BIGSERIAL (bigint)	NO	nextval('providers_id_seq')
user_id	BIGINT	NO	
display_name	VARCHAR(100)	NO	
headline	VARCHAR(200)	NO	
bio	TEXT	YES	

PK: providers_pkey (id)

FKs: providers_user_id_fkey: user_id → users(id) ON DELETE CASCADE

UNIQUE: providers_user_id_key (user_id)

CHECK: none

Indexes: providers_pkey, providers_user_id_key (unique B-tree indexes backing the constraints)

services

Column	Type	Nullable	Default
id	BIGSERIAL (bigint)	NO	nextval('services_id_seq')
name	VARCHAR(100)	NO	
description	TEXT	YES	
duration_minutes	INT	NO	
price_cents	INT	NO	

PK: services_pkey (id)

FKs: none

UNIQUE: services_name_key (name)

CHECK: services_duration_minutes_check: duration_minutes > 0; services_price_cents_check: price_cents >= 0

Indexes: services_pkey, services_name_key (unique B-tree indexes backing the constraints)

availability_slots

Column	Type	Nullable	Default
id	BIGSERIAL (bigint)	NO	nextval('availability_slots_id_seq')
provider_id	BIGINT	NO	
service_id	BIGINT	NO	
start_time	TIMESTAMP	NO	
end_time	TIMESTAMP	NO	
status	VARCHAR(20)	NO	'OPEN'
version	INT	NO	0

PK: availability_slots_pkey (id)

FKs: availability_slots_provider_id_fkey: provider_id → providers(id) ON DELETE CASCADE; availability_slots_service_id_fkey: service_id → services(id)

UNIQUE: uq_slot_provider_start (provider_id, start_time); uq_slot_id_service (id, service_id)

CHECK: availability_slots_status_check: status IN ('OPEN', 'BOOKED', 'REMOVED'); ck_slot_time_order: end_time > start_time

Indexes: availability_slots_pkey, uq_slot_provider_start, uq_slot_id_service (unique B-tree indexes backing the constraints); ix_slots_open_start: B-tree on (status, start_time)

appointments

Column	Type	Nullable	Default
id	BIGSERIAL (bigint)	NO	nextval('appointments_id_seq')
slot_id	BIGINT	NO	
service_id	BIGINT	NO	
customer_id	BIGINT	NO	
status	VARCHAR(20)	NO	'BOOKED'
notes	TEXT	YES	
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP
cancelled_at	TIMESTAMP	YES	

PK: appointments_pkey (id)

FKs: fk_appt_slot_service: (slot_id, service_id) → availability_slots(id, service_id); appointments_customer_id_fkey: customer_id → users(id)

UNIQUE: none as table constraints (see partial unique index below)

CHECK: appointments_status_check: status IN ('BOOKED', 'CANCELLED', 'COMPLETED')

Indexes: appointments_pkey (unique B-tree index backing the PK); uq_appt_active_slot: UNIQUE B-tree on (slot_id) WHERE status = 'BOOKED' (double-booking guard); ix_appt_customer: B-tree on (customer_id)

Appendix B. schema.sql

Source: src/main/resources/schema.sql

```
-- Interview Scheduler schema (PostgreSQL). All SQL hand-written; no ORM.
-- Milestone 1: tables are dropped and recreated on every startup so seed data is fresh.
```

```
DROP TABLE IF EXISTS appointments;
DROP TABLE IF EXISTS availability_slots;
DROP TABLE IF EXISTS services;
DROP TABLE IF EXISTS providers;
DROP TABLE IF EXISTS users;

-- Every account. Role decides which endpoints the account may use.
CREATE TABLE users (
  id          BIGSERIAL    PRIMARY KEY,
  username    VARCHAR(50)  NOT NULL UNIQUE,
  password_hash VARCHAR(100) NOT NULL,          -- BCrypt hash, never plaintext
  full_name   VARCHAR(100) NOT NULL,
  email       VARCHAR(255) NOT NULL UNIQUE,
  role        VARCHAR(20)  NOT NULL CHECK (role IN ('CUSTOMER', 'PROVIDER')),
  created_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```

-- Provider profile (a mentor / interviewer). 1:1 with a PROVIDER user.
CREATE TABLE providers (
  id          BIGSERIAL    PRIMARY KEY,
  user_id     BIGINT       NOT NULL UNIQUE REFERENCES users (id) ON DELETE CASCADE,
  display_name VARCHAR(100) NOT NULL,
  headline    VARCHAR(200) NOT NULL,          -- e.g. "Senior SWE, 8 yrs backend"
  bio         TEXT
);

-- Bookable service types (resume review, mock interviews, ...).
CREATE TABLE services (
  id          BIGSERIAL    PRIMARY KEY,
  name        VARCHAR(100) NOT NULL UNIQUE,
  description  TEXT,
  duration_minutes INT      NOT NULL CHECK (duration_minutes > 0),
  price_cents  INT          NOT NULL CHECK (price_cents >= 0)
);

-- An open time window a provider offers for one service.
CREATE TABLE availability_slots (
  id          BIGSERIAL    PRIMARY KEY,
  provider_id BIGINT       NOT NULL REFERENCES providers (id) ON DELETE CASCADE,
  service_id  BIGINT       NOT NULL REFERENCES services (id),
  start_time  TIMESTAMP    NOT NULL,
  end_time    TIMESTAMP    NOT NULL,
  status      VARCHAR(20)  NOT NULL DEFAULT 'OPEN' CHECK (status IN ('OPEN', 'BOOKED', 'REMOVED')),
  version     INT          NOT NULL DEFAULT 0,      -- optimistic-lock counter (used in Milestone 2)
  CONSTRAINT ck_slot_time_order CHECK (end_time > start_time),
  -- A provider cannot publish two slots that start at the same moment.
  CONSTRAINT uq_slot_provider_start UNIQUE (provider_id, start_time),
  -- Target for the composite FK below (lets appointments prove slot/service agree).
  CONSTRAINT uq_slot_id_service UNIQUE (id, service_id)
);

CREATE INDEX ix_slots_open_start ON availability_slots (status, start_time);

-- A booking: links a customer, a slot and the slot's service.
CREATE TABLE appointments (
  id          BIGSERIAL    PRIMARY KEY,
  slot_id     BIGINT       NOT NULL,
  service_id  BIGINT       NOT NULL,
  customer_id BIGINT       NOT NULL REFERENCES users (id),
  status      VARCHAR(20)  NOT NULL DEFAULT 'BOOKED' CHECK (status IN ('BOOKED', 'CANCELLED', 'COMPLETED')),
  notes       TEXT,        -- e.g. target role, resume link
  created_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  cancelled_at TIMESTAMP,
  -- The appointment's service must be the slot's service.
  CONSTRAINT fk_appt_slot_service FOREIGN KEY (slot_id, service_id)
    REFERENCES availability_slots (id, service_id)
);

-- DOUBLE-BOOKING GUARD: at most one *active* (BOOKED) appointment per slot.
-- Cancelled rows stay for history and don't block rebooking the slot.
CREATE UNIQUE INDEX uq_appt_active_slot ON appointments (slot_id) WHERE status = 'BOOKED';

CREATE INDEX ix_appt_customer ON appointments (customer_id);

```

Appendix C. Screenshots

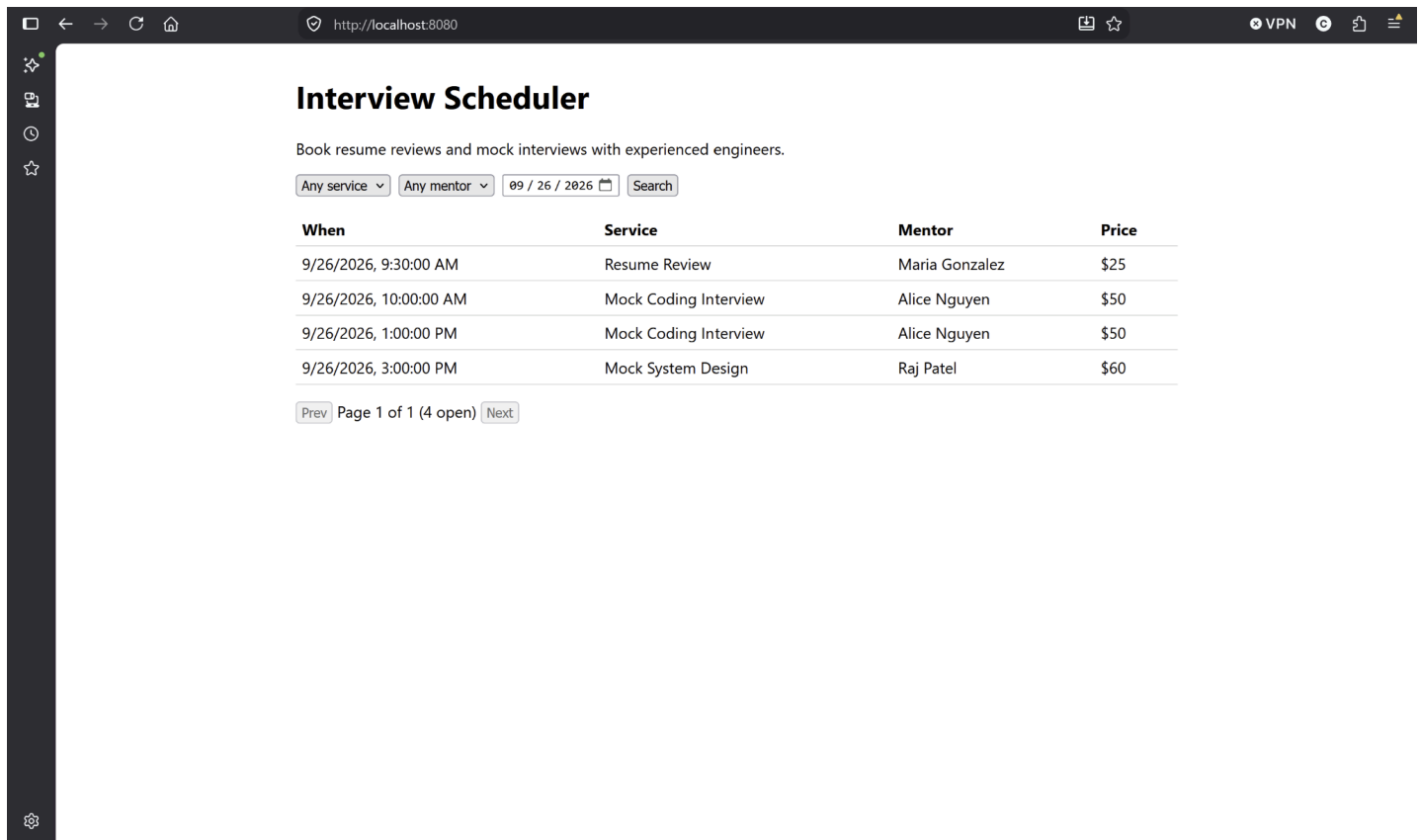


Figure C1. Home Page of scheduler in Milestone 1

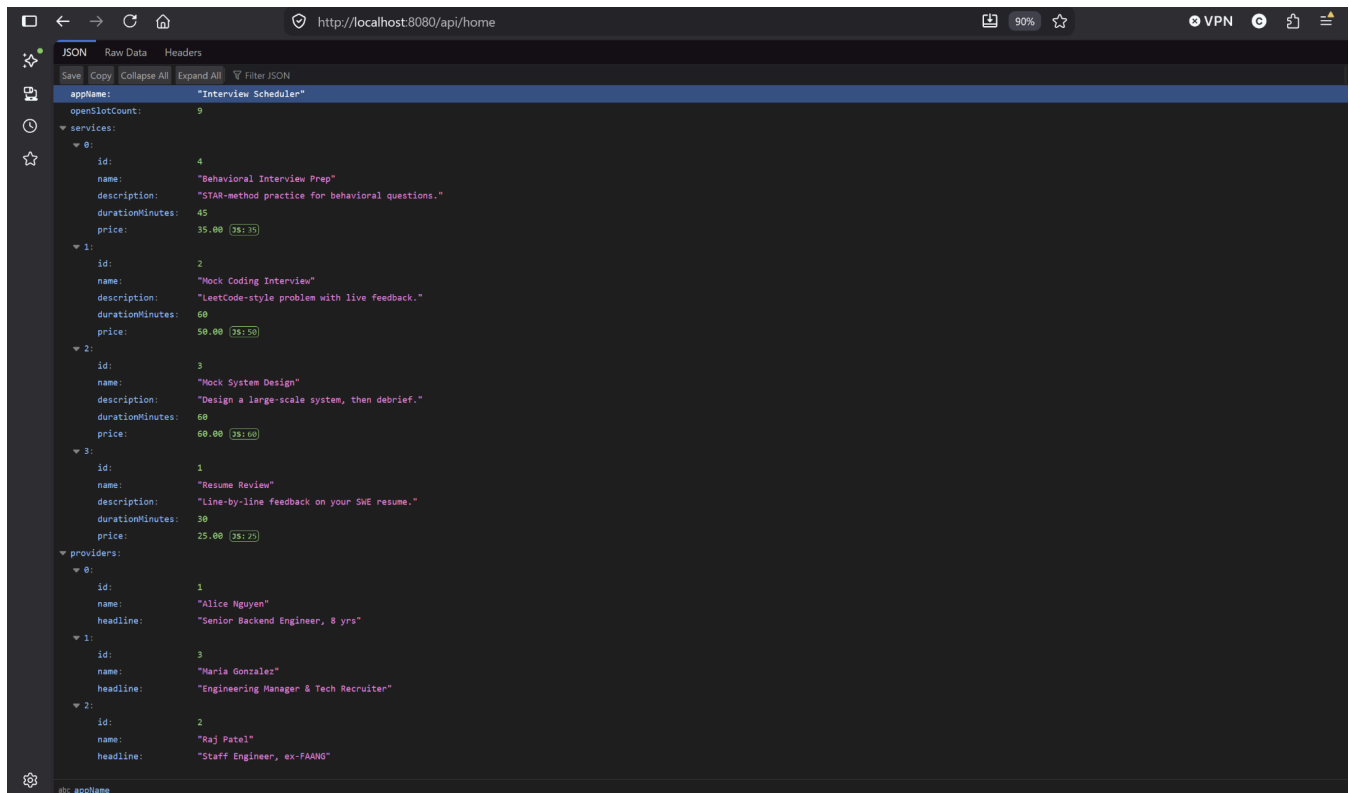


Figure C2. `/api/home` JSON

```

JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
▼ Items:
0:
  id: 8
  providerId: 3
  providerName: "Maria Gonzalez"
  serviceId: 1
  serviceName: "Resume Review"
  durationMinutes: 30
  price: 25.00 (25.25)
  startTime: "2026-09-26T09:30:00"
  endTime: "2026-09-26T10:00:00"
1:
  id: 1
  providerId: 1
  providerName: "Alice Nguyen"
  serviceId: 2
  serviceName: "Mock Coding Interview"
  durationMinutes: 60
  price: 50.00 (25.50)
  startTime: "2026-09-26T10:00:00"
  endTime: "2026-09-26T11:00:00"
2:
  id: 2
  providerId: 1
  providerName: "Alice Nguyen"
  serviceId: 2
  serviceName: "Mock Coding Interview"
  durationMinutes: 60
  price: 50.00 (25.50)
  startTime: "2026-09-26T13:00:00"
  endTime: "2026-09-26T14:00:00"
3:
  id: 4
  providerId: 2
  providerName: "Raj Patel"
  serviceId: 3
  serviceName: "Mock System Design"
  durationMinutes: 60
  price: 60.00 (25.60)
  startTime: "2026-09-26T15:00:00"
  endTime: "2026-09-26T16:00:00"
4:
  id: 3
  providerId: 1
  providerName: "Alice Nguyen"
  serviceId: 1
  serviceName: "Resume Review"
  durationMinutes: 30
  {} items > ( ) 6 > 40 serviceName

```

```

JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
▼ Items:
5:
  id: 9
  providerId: 3
  providerName: "Maria Gonzalez"
  serviceId: 4
  serviceName: "Behavioral Interview Prep"
  durationMinutes: 45
  price: 35.00 (25.35)
  startTime: "2026-09-27T09:00:00"
  endTime: "2026-09-27T09:30:00"
6:
  id: 5
  providerId: 2
  providerName: "Raj Patel"
  serviceId: 3
  serviceName: "Mock System Design"
  durationMinutes: 60
  price: 60.00 (25.60)
  startTime: "2026-09-28T15:00:00"
  endTime: "2026-09-28T16:00:00"
7:
  id: 6
  providerId: 2
  providerName: "Raj Patel"
  serviceId: 2
  serviceName: "Mock Coding Interview"
  durationMinutes: 60
  price: 50.00 (25.50)
  startTime: "2026-09-29T10:00:00"
  endTime: "2026-09-29T11:00:00"
8:
  id: 10
  providerId: 3
  providerName: "Maria Gonzalez"
  serviceId: 4
  serviceName: "Behavioral Interview Prep"
  durationMinutes: 45
  price: 35.00 (25.35)
  startTime: "2026-09-30T14:00:00"
  endTime: "2026-09-30T14:45:00"
page: 0
size: 10
totalItems: 9
totalPages: 1
{} items > ( ) 4 > 40 serviceName

```

Figure C3. /api/slots JSON